

Práctica 7



Instalar Dependencias

npm init

```
PS C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica 7> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (practica-7)
version: (1.0.0)
description:
entry point: (insertMongoDB.js)
test command:
git repository:
keywords:
author:
```

npm install express

```
PS C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica 7> npm install express
added 65 packages, and audited 66 packages in 8s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

npm install mongodb

```
PS C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica 7> npm install mongodb
added 12 packages, and audited 78 packages in 8s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

nslookup cluster0.5qnxasc.mongodb.net

```
PS C:\Users\nadxe\Desktop\UPIITA\Aplicaciones Distribuidas\practica 7> nslookup cluster0.5qnxasc.mongodb.net
Servidor: pc-87-006.upiita.ipn.mx
Address: 148.204.87.6

Nombre: cluster0.5qnxasc.mongodb.net
```

Objetivo de la práctica

Se implementó un servidor web basado en Node.js y Express que facilita la creación de una API REST conectada a MongoDB. El programa ilustra cuatro formas fundamentales de interacción con el servidor: el uso de parámetros en la URL, en la ruta, en el cuerpo de la solicitud y la ejecución de operaciones de almacenamiento de datos.

Inicialización del servidor

El servidor comienza importando las librerías necesarias y creando la aplicación Express.

```
var express = require("express");  
  
var app = express(); const { MongoClient } = require("mongodb");
```

Aquí se crea el objeto 'app', el cual actúa como contenedor de todos los servicios REST que se definirán posteriormente. Posteriormente se habilitan los middlewares para recibir datos en formato

```
JSON. app.use(express.json());  
  
app.use(express.urlencoded({ extended: true }));
```

Conectamos la base de datos

Para almacenar información, el sistema establece una conexión con MongoDB mediante el cliente oficial.

```
async function connectDB()  
{ const uri = "mongodb+srv://usuario:password@cluster.mongodb.net/?appName=Cluster0";  
  
  client = new MongoClient(uri);  
  
  await client.connect(); }
```

Una vez establecida la conexión, se define la base de datos y la colección que se utilizarán.

```
function prepareDB() { dbName = "myDatabase";
```

```
collectionName = "recipes";

database = client.db(dbName);

collection = database.collection(collectionName); }
```

Inicio del servidor

El servidor comienza a escuchar solicitudes HTTP en el puerto 3000.

```
app.listen(3000, function () { console.log("Aplicación ejemplo, escuchando el puerto 3000!");
connectDB();

prepareDB(); });
```

Endpoint raíz

Este endpoint permite verificar que el servidor se encuentra activo.

```
app.get("/", async function (request, response)

{ r = { message: "Nothing to send", }; response.json(r); });
```

Petición: GET http://localhost:3000

Servicio con parámetros en la URL

```
app.get("/serv001", async function (req, res) { const user_id = req.query.id;

const token = req.query.token; const geo = req.query.geo;

r = { user_id: user_id, token: token, geo: geo, };

res.json(r); });
```

Ejemplo de solicitud: GET http://localhost:3000/serv001?id=123&token=abc&geo=19.43,-99.13

Servicio con parámetros en el body (POST)

```
app.post("/serv002", async function (req, res) { const user_id = req.body.id;
```

```
const token = req.body.token;

const geo = req.body.geo;

r = { user_id: user_id, token: token, geo: geo, };

res.json(r);

});
```

Este endpoint recibe los datos en formato JSON dentro del cuerpo de la petición.

Servicio con parámetros en la ruta

```
app.post("/serv003/:info", async function (req, res) { const info = req.params.info;

let r = { info: info };

res.json(r);

});
```

Ejemplo de solicitud: POST <http://localhost:3000/serv003/987654>

Inserción de documentos en MongoDB

```
p.post("/receipt/insert", async function (req, res)

{ const recipes = [

{ name: "elotes cocidos", ingredients:

[ "corn", "mayonnaise", "cotija cheese", "sour cream", "lime", ],

prepTimeInMinutes: 35, }, ],
```

Conclusión

El sistema implementa un servidor REST capaz de recibir solicitudes HTTP mediante distintos métodos y formatos de entrada. Se verificó que los endpoints responden correctamente a peticiones GET y POST, permitiendo recibir parámetros desde la URL, desde el cuerpo de la solicitud o desde la ruta del endpoint. Asimismo, el servidor puede interactuar con MongoDB para almacenar documentos.

```
_id: ObjectId('69b1856b62741b31d748ecc3')  
name: "elotes cocidos"  
▸ ingredients: Array (5)  
prepTimeInMinutes: 35
```